

PredCLOCK: Machine Learning-Enhanced CLOCK Cache Replacement Policy for Improved Efficiency

Muhammad Haekal Muhyidin Al-Araby
Department of Computer Engineering
Institut Teknologi Sepuluh Nopember
Surabaya, Indonesia
5024221030@student.its.ac.id

Reza Fuad Rachmadi, S.T., M.T., Ph.D
Department of Computer Engineering
Institut Teknologi Sepuluh Nopember
Surabaya, Indonesia
fuad@its.ac.id

Dr. Arief Kurniawan, S.T., M.T.
Department of Computer Engineering
Institut Teknologi Sepuluh Nopember
Surabaya, Indonesia
arifku@ee.its.ac.id

Abstract—This research focuses on the development and analysis of PredCLOCK, a machine learning-based enhancement to the CLOCK cache eviction algorithm. While the standard CLOCK algorithm is widely used due to its low overhead, it suffers from the “blind promotion” phenomenon, where objects with low reuse probability (“one-hit wonders”) are unnecessarily promoted, leading to cache pollution and wasted write operations. This study aims to mitigate these inefficiencies by integrating a supervised learning model into the eviction decision process to filter promotions based on access patterns consistent with Zipfian distributions. Simulations were conducted using a custom high-performance framework based on *LibCacheSim* under trace replay scenarios. The quantitative evaluation demonstrates that PredCLOCK achieves a consistent reduction in cache promotions of approximately 8–10% across various cache sizes compared to the standard CLOCK baseline. Furthermore, the system yields an improvement in cache efficiency with a reduction in Miss Ratio (up to 0.20%) in high-pressure environments (1%–20% cache size). Although the write efficiency gains fall short of the theoretical 43% upper bound established by the *Offline CLOCK* oracle, the proposed approach successfully optimizes the trade-off between cache hit rates and hardware endurance (write reductions) without requiring future knowledge.

Index Terms—Cache Replacement, CLOCK Algorithm, Machine Learning, Supervised Learning, Write Endurance, Zipfian Distribution.

I. INTRODUCTION

The fundamental challenge in modern memory hierarchies is bridging the latency gap between processing units and backing stores, a problem famously characterized as the “Memory Wall” [1]. To mitigate this, systems employ multi-level cache hierarchies where the efficiency of the Last Level Cache (LLC) or software-managed cache dictates overall system performance [2]. In these environments, the primary goal of a replacement policy is to maximize the hit ratio by accurately identifying and retaining the workload’s active “working set” [3].

While Least Recently Used (LRU) algorithms generally provide high hit ratios, they incur significant synchronization overhead in multi-core environments due to the need for

fine-grained locking on every access [4]. Consequently, the CLOCK algorithm and its concurrent variants, such as NbQ-CLOCK, have become the standard for high-throughput systems [5]. However, CLOCK relies on a coarse-grained “blind promotion” mechanism: any accessed item is given a “second chance” (reference bit set to 1). This simplicity leaves CLOCK vulnerable to cache pollution caused by scans or “one-hit wonders”—items accessed once and never again [6]. When these low-utility items are blindly promoted, they displace high-utility data, degrading the cache’s effective capacity and overall hit ratio.

Addressing this inefficiency requires distinguishing between frequency and mere recency. Advanced heuristics like CAR (Clock with Adaptive Replacement) [7] and CLOCK-Pro [6] attempt to filter these items using history lists and test segments. However, modern workloads, particularly web traffic characterized by Zipf-like distributions, exhibit complex access patterns that static heuristics often fail to capture optimally [8]. Recent studies suggest that deciding what to keep (promotion policy) is often more critical for efficiency than deciding what to evict [9], [10].

In this paper, we introduce PredCLOCK (Predictive CLOCK), a mechanism to increase the promotion efficiency of the CLOCK algorithm. Rather than granting a “second chance” to every accessed item, PredCLOCK utilizes a lightweight Supervised Learning model to predict the reuse probability of an item at the moment of access. This allows the system to filter useless promotions, effectively acting as a gatekeeper that protects the cache from pollution. By leveraging recent advances in efficient ML inference [11], we demonstrate that PredCLOCK achieves higher cache utilization efficiency and robustness against scan-heavy workloads compared to standard CLOCK implementations.

II. RELATED WORK

The landscape of cache replacement policies has evolved from simple static heuristics to complex, adaptive algorithms,

and recently, to machine learning-augmented systems. Research typically optimizes for two often conflicting goals: maximizing the hit ratio (scan resistance) and minimizing implementation overhead (concurrency and scalability).

A. Classical Replacement and Scalability

The Least Recently Used (LRU) algorithm has historically been the baseline for replacement policies due to its exploitation of temporal locality [3]. However, exact LRU requires a doubly-linked list update on every cache access. In modern multicore processors, the fine-grained locking required to protect this structure results in severe contention, limiting system throughput [4].

To address scalability, the CLOCK algorithm approximates LRU using a circular buffer and reference bits, eliminating the need for list manipulation on hits. Eads et al. further refined this for high-concurrency environments with **NbQ-CLOCK** (Non-blocking Queue CLOCK) [5]. NbQ-CLOCK replaces the circular buffer with a concurrent, lock-free queue, allowing for high throughput. However, both standard CLOCK and NbQ-CLOCK suffer from “blind promotion,” where any accessed item is granted a second chance. This makes them susceptible to cache pollution during scan operations (sequences of “one-hit wonders”), which displace high-utility data [6].

B. Scan Resistance and Advanced Heuristics

To mitigate cache pollution caused by scans and Zipfian long-tail distributions [8], several heuristics were developed to separate frequency from recency.

LIRS (Low Inter-reference Recency Set) utilizes the Inter-Reference Recency (IRR) between the last two accesses to predict future utility, effectively filtering out items with weak locality [12]. **ARC** (Adaptive Replacement Cache) dynamically balances recency and frequency by maintaining two lists and adapting their size based on workload shifts [13].

Building on these concepts, **CAR** (Clock with Adaptive Replacement) adapts the self-tuning benefits of ARC to the more efficient CLOCK structure [7]. Similarly, **CLOCK-Pro** introduces a “test period” for cold pages; items are not promoted to the protected segment immediately upon insertion but must demonstrate reuse within a test window [6]. While these algorithms improve scan resistance, they rely on static rules that may not adapt quickly enough to the complex, non-stationary patterns of modern AI and web workloads [14].

C. Machine Learning in Caching

The complexity of modern access patterns has motivated the integration of Machine Learning (ML) into cache management. **LeCaR** (Learning Cache Replacement) employs Reinforcement Learning (RL) using regret minimization to dynamically weight two underlying policies (LRU and LFU) [15]. While LeCaR adapts to phase changes, it essentially selects between existing static policies rather than predicting individual item utility.

Recent work has focused on “lightweight” ML to minimize inference latency in real-time systems. Berger et al. demonstrated that object-level features (e.g., size, type) could be

used to predict hit probabilities in CDNs [16]. More recently, the importance of the *promotion* path—deciding what enters the cache—has gained attention. Chen et al. explored **lazy promotion**, showing that deferring promotion decisions can significantly reduce the churn caused by low-utility items [9]. Similarly, modern FIFO-based approaches like **S3-FIFO** argue that simple queues with intelligent promotion thresholds can outperform complex LRU variants [17].

D. The PredCLOCK Approach

Different from RL-based policy selection, **PredCLOCK** utilizes a Supervised Learning model to act as an intelligent gatekeeper. By predicting reuse probability upon access, we explicitly target the inefficiency of blind promotion found in CLOCK and NbQ-CLOCK. This approach aligns with recent findings that optimizing promotion efficiency is often more impactful on hit ratios than optimizing eviction policies [9], [18].

III. SYSTEM DESIGN

This chapter details the architectural design and implementation strategies of the proposed *PredCLOCK* system. The system is designed as a hybrid simulation environment that bridges high-performance cache simulation (written in C++) with predictive machine learning models (trained in Python) to optimize cache eviction decisions.

A. System Architecture Overview

The system architecture is constructed to ensure minimal latency overhead while maintaining high prediction accuracy. As illustrated in Figure 1, the system comprises three main components: the Data Ingestion Layer (Trace Replay), the Core Simulation Engine (LibCacheSim), and the Inference Engine (ONNX Runtime).

The **Data Ingestion Layer** processes synthetic Zipfian traces or real-world datasets, parsing object IDs and access timestamps. These are fed into the **Core Simulation Engine**, which implements the cache logic. Unlike standard CLOCK implementations, our engine includes a hook to the **Inference Engine** whenever a promotion decision is required. We utilize *ONNX Runtime* [19] to execute the model within the C++ environment. This approach mitigates the interoperability costs and latency overhead typically associated with Python-C++ context switching during runtime [20].

B. Program Workflow

The research methodology follows a sequential workflow divided into offline training and online inference phases. Figure 2 outlines the complete flow of the program.

The workflow consists of four distinct stages:

- 1) **Trace Generation and Baseline Simulation:** Initial traces are run through a standard CLOCK simulator to establish baseline metrics such as hit ratio and eviction count.
- 2) **Labeling (Offline Oracle):** We utilize an *Offline CLOCK* algorithm with future knowledge to label

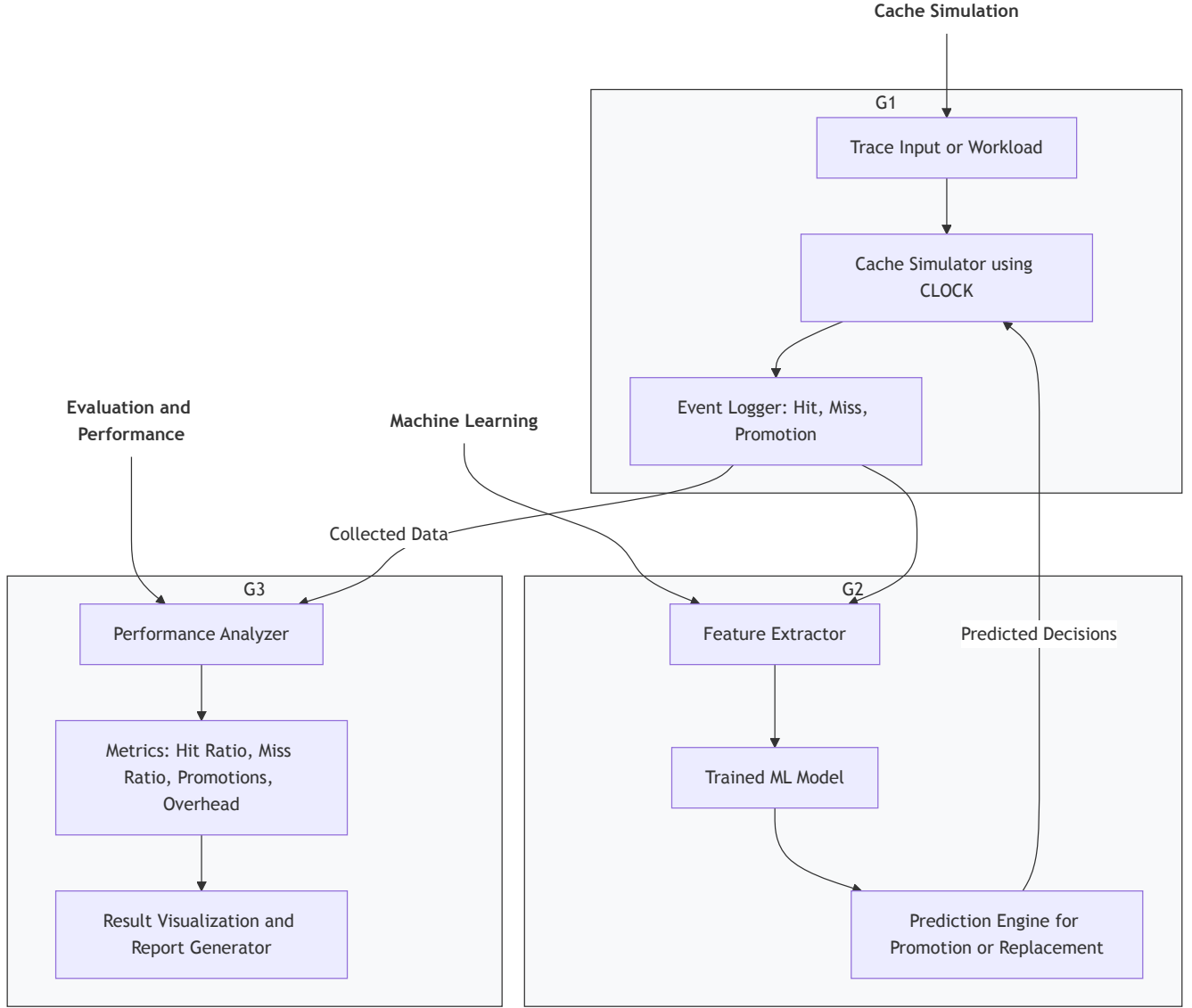


Fig. 1: High-level system architecture showing the interaction between the Trace Replay mechanism, the Cache Simulator, and the ML Inference Engine via ONNX.

dataset entries. Objects are labeled based on their utility: “Wasted” (promoted but never hit before eviction) or “Reused” (promoted and subsequently hit) [14].

- 3) **Training:** A lightweight neural network is trained using the labeled dataset to recognize patterns in frequency and recency.
- 4) **Inference Integration:** The trained model is exported to the ONNX format and loaded into the modified simulator for the final evaluation.

C. Machine Learning Model Design

The core of the decision-making process is a Supervised Learning model designed to classify cache objects based on their likelihood of future reuse.

1) *Model Architecture:* We employ a Multi-Layer Perceptron (MLP) defined using the Keras API [21]. As shown in Figure 3, the model takes a vector of features representing object history (e.g., frequency, reuse distance) and processes them through dense hidden layers with ReLU activation.

The output layer utilizes a *Sigmoid* activation function, producing a scalar probability score between 0 and 1. This score represents the probability that promoting the object will result in a cache hit.

2) *Training Pipeline:* The training process, depicted in Figure 4, involves preprocessing raw trace logs into feature vectors. To address class imbalance—where “wasted” promotions often outnumber successful ones in specific workloads—we apply class weighting during the training phase.

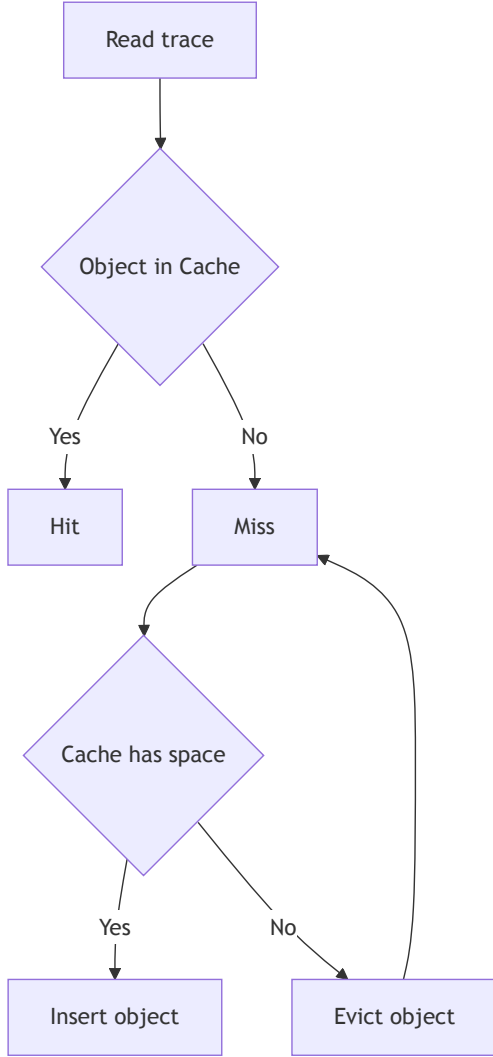


Fig. 2: Program workflow diagram illustrating the stages from Trace Generation, Labeling via Offline CLOCK, Model Training, to Final Simulation.

D. Implementation and Decision Logic

The integration of the model into the CLOCK algorithm alters the standard eviction behavior to implement a “Lazy Promotion” strategy [9]. Figure 5 details the logic flow during a cache eviction candidate check.

In a standard CLOCK algorithm, an object with a reference bit of 1 is essentially guaranteed a “second chance” and remains in the cache. In our PredCLOCK design, we intercept this process:

- 1) When the clock hand points to an object with `ref_bit = 1`, the system queries the ONNX model.
- 2) If the model predicts a low probability of reuse ($P < \text{Threshold}$), the promotion is denied. The object is treated as if `ref_bit = 0` and is evicted immediately.
- 3) If the model predicts high reuse, the standard CLOCK

TABLE I: Synthetic Zipfian Dataset Specifications

Parameter	Value
Total Requests	10,000,000
Unique Objects	1,000,000
Zipf Parameter (α)	1.0
Trace Format	oracleGeneral (Binary)
Data per Entry	16 bytes (Time + ID + Size)
Distribution Type	Stationary

logic proceeds: the reference bit is reset to 0, and the hand moves to the next candidate.

This mechanism ensures that the cache is not polluted by “one-hit-wonder” objects that appear frequently in scan-heavy workloads, addressing the specific weaknesses of CLOCK identified in previous works [6].

IV. EXPERIMENTAL SETUP

To rigorously evaluate the proposed *PredCLOCK* (ML-CLOCK) mechanism, we developed a custom simulation environment capable of trace replay and machine learning inference. This section details the hardware infrastructure, dataset characteristics, baseline algorithms, and performance metrics used in our evaluation.

A. Simulation Environment

The experiments were conducted on a high-performance server designed to handle large-scale trace simulations and parallel processing. The specific specifications are chosen to ensure the entire trace and cache structures reside in memory, minimizing I/O bottlenecks during the simulation [22].

- **Hardware:** The testbed consists of an Intel Xeon Silver 4114 processor (40 logical cores at 3.00 GHz) and 192 GiB of RAM.
- **Software Stack:** The simulator is implemented in C++ for efficiency, utilizing *LibCacheSim* [22] as the foundational library for cache modeling and trace parsing.
- **Inference Engine:** For the ML-based eviction logic, we integrated *ONNX Runtime* [19] directly into the C++ environment. This allows the system to execute models trained in Python (via Keras/TensorFlow [21], [23]) with minimal latency overhead during the eviction decision process [20].

B. Workload Datasets

To ensure reproducibility and isolate the impact of access patterns, we utilized synthetic traces following a Zipfian distribution. Zipfian distributions accurately model real-world web traffic, where a small fraction of content accounts for the majority of requests [8].

The datasets were generated procedurally using *LibCacheSim* utilities in the `oracleGeneral` binary format. Table I summarizes the workload parameters. We configured the Zipfian parameter (α) to 1.0, representing a standard skew found in web workloads. The stationary nature of these traces allows us to evaluate the algorithm’s steady-state performance without the noise of phase changes.



Fig. 3: Architecture of the Feed-Forward Neural Network used for eviction prediction. The final layer uses a Sigmoid activation to output a reuse probability score.

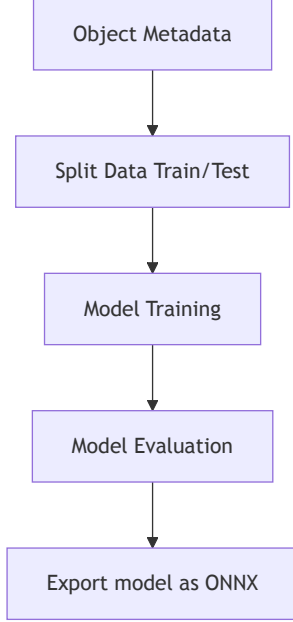


Fig. 4: Machine Learning training pipeline, detailing feature extraction, data splitting, and model export to ONNX.

C. Baselines for Comparison

We compare the performance of our proposed *PredCLOCK* against three distinct baselines to evaluate its efficiency and effectiveness:

- 1) **Standard CLOCK:** The unmodified CLOCK algorithm serves as the primary baseline. It operates purely on recency using a circular buffer and reference bits. It is known to suffer from scan resistance issues, where one-time access objects flush useful data.
- 2) **Offline CLOCK (Theoretical Upper Bound):** This algorithm acts as an oracle with perfect knowledge of future accesses. It performs an initial scan of the trace to identify promotions that result in hits. If a promotion is deemed "Wasted" (the object is evicted before being hit), Offline CLOCK suppresses the promotion. This represents the theoretical maximum efficiency achievable [24].
- 3) **Optimal Zipf Promotion:** A heuristic-based oracle that promotes objects based solely on their global popularity ranking within the Zipfian distribution. This baseline highlights the value of global frequency knowledge versus local access patterns.

D. Evaluation Metrics

To measure the effectiveness of the cache replacement strategies, we focus on two key metrics:

- **Miss Ratio:** The fraction of requests that are not served from the cache. A lower miss ratio indicates better cache utility.
- **Promotion Rate:** The total number of items promoted from the probationary state to the protected state (or inserted into the main cache). Reducing unnecessary promotions is critical for minimizing memory writes, which is particularly important for extending the lifespan of SSD-based caches [25], [26].

Experiments were conducted across various cache sizes, ranging from 1% to 40% of the total unique object footprint, to analyze performance under different levels of cache pressure.

V. EVALUATION

This section presents the quantitative evaluation of the proposed **PredCLOCK** system. The performance is analyzed in two phases: first, a comparative analysis against standard and theoretical baselines to establish overall efficacy, and second, a deep dive into model configuration and parameter tuning to understand the impact of loss functions on cache behavior.

A. Overall Performance Comparison

To evaluate the effectiveness of the proposed solution, *PredCLOCK* was benchmarked against three references: Standard CLOCK (Baseline), Optimal Zipf Promotion (Heuristic Oracle), and Offline CLOCK (Theoretical Upper Bound). The experiments were conducted across four cache sizes: 1%, 10%, 20%, and 40% of the total unique object footprint.

Table II summarizes the performance trade-offs. Negative values in "Delta Miss Ratio" indicate improvement (reduction in misses), while positive values indicate regression.

1) *Analysis of Cache Efficiency:* The results highlight a clear non-linear performance pattern. In environments with high memory pressure (cache sizes 1%–20%), **PredCLOCK** provides a "double benefit": a modest reduction in Miss Ratio (-0.05% to -0.20%) and a consistent reduction in memory writes (promotions). This confirms that the model successfully identifies "one-hit wonders" (wasted objects) during scan phases, preventing them from polluting the cache [8], [27].

However, at the 40% cache size, a performance regression (+0.15%) is observed. This indicates that the static decision threshold becomes *over-selective* when cache space is abundant. The model continues to reject marginal objects that could

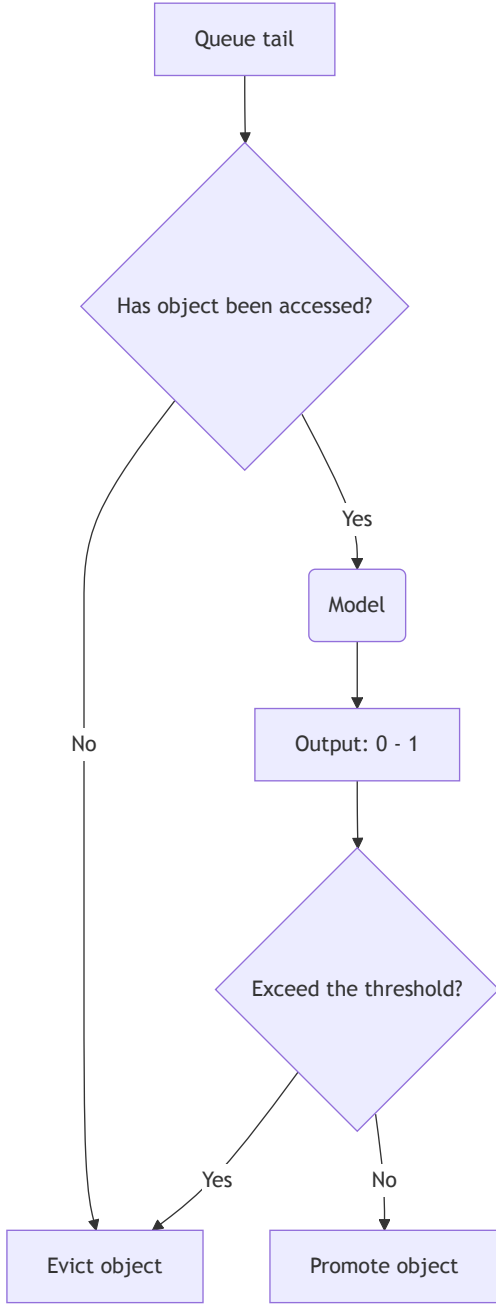


Fig. 5: Decision logic for the ML-Enhanced CLOCK Algorithm. The model acts as a filter for promotions, denying the “second chance” to low-utility objects.

have been stored without penalty, leading to valid data being discarded.

2) *Analysis of Write Efficiency vs. Theoretical Bounds:* A key contribution of PredCLOCK is the reduction of memory write operations. Across all tested scenarios, the system reduces the promotion rate by approximately **8–10%**. While this is significant for extending the endurance of flash-based storage (SSDs) [25], [26], it falls significantly short of the

TABLE II: Performance Comparison: PredCLOCK vs. Baselines

Size	Method	Delta Miss	Prom. Red.
1%	PredCLOCK	-0.05%	10%
	Optimal Zipf	-0.20%	10%
	Offline CLOCK	-0.35%	40%
10%	PredCLOCK	-0.10%	10%
	Optimal Zipf	-0.90%	13%
	Offline CLOCK	-1.80%	43%
20%	PredCLOCK	-0.20%	10%
	Optimal Zipf	-1.67%	15%
	Offline CLOCK	-3.50%	43%
40%	PredCLOCK	+0.15%	8%
	Optimal Zipf	-3.30%	17%
	Offline CLOCK	-8.00%	43%

43% reduction achieved by *Offline CLOCK*.

This gap highlights the limitations of local-feature prediction compared to the global future knowledge possessed by the *Offline CLOCK* oracle [9]. PredCLOCK remains conservative, prioritizing cache safety (Hit Ratio) over maximum write reduction.

B. Model Parameter Tuning and Configuration

The performance of PredCLOCK is highly sensitive to the handling of class imbalance. In Zipfian workloads ($\alpha \approx 1.0$), the ratio of “wasted” objects (accessed once) to “reused” objects is approximately 80:20. Without intervention, machine learning models tend to bias towards the majority class. We evaluated two distinct model configurations: Conservative (Weighted) and Aggressive (Unweighted).

1) *Configuration A: Conservative Model (Weighted Loss):* This variant was trained using a custom weighted loss function (ratio 2:1), explicitly penalizing False Positives (incorrectly evicting a popular object) more heavily than False Negatives. A high decision threshold (0.8) was applied to ensure objects are only rejected when the model is highly confident.

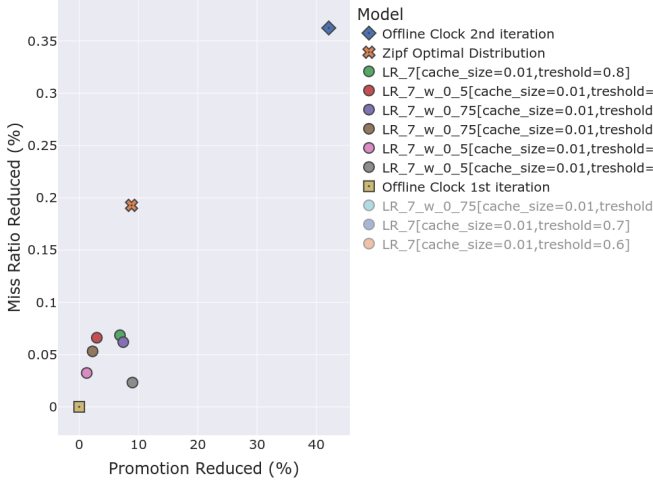
The results, visualized in Figures 6a through 6b, show that this configuration successfully mimics the safety of the standard CLOCK algorithm.

- 1) **Performance stability:** The Miss Ratio closely tracks the baseline, avoiding service degradation.
- 2) **Controlled Efficiency:** The promotion reduction is capped at roughly 10%, representing a safe trade-off between hardware endurance and cache hit rates.

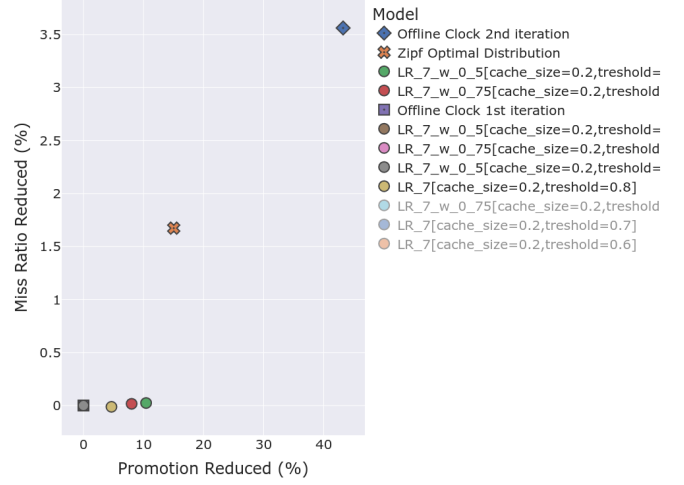
2) *Configuration B: Aggressive Model (Unweighted Loss):* The Aggressive Model was trained without class balancing. As predicted by the class imbalance problem [28], the model converged to a local minimum by predicting the majority class (“Wasted”) too frequently.

The results, shown in Figures 7a through 7b, demonstrate a systemic failure of this strategy:

- **Service Degradation:** A spike in Miss Ratio (negative values on the Y-axis graphs) of approximately 2% was observed compared to Standard CLOCK.

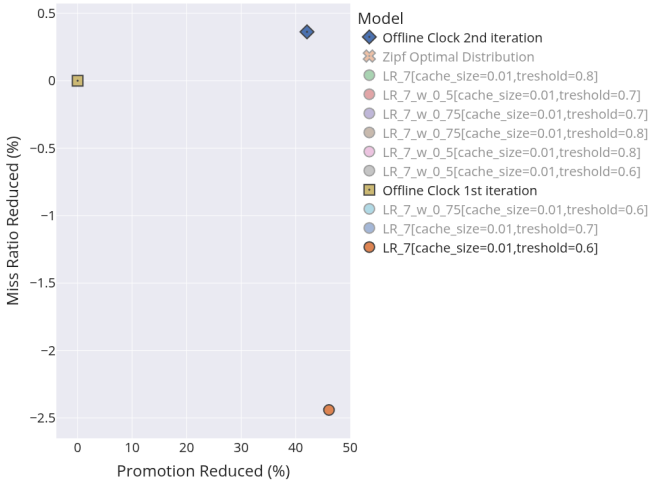


(a) 1% Cache Size

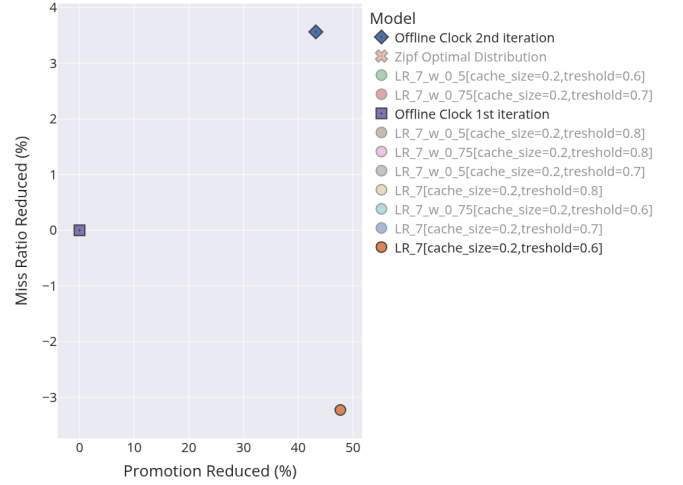


(b) 20% Cache Size

Fig. 6: Conservative Model performance across cache sizes. Note the slight improvement at 1% and the regression at 40% due to over-selection.



(a) 1% Cache Size



(b) 20% Cache Size

Fig. 7: Aggressive Model performance across cache sizes. Negative values indicate performance degradation and thrashing.

- **Cache Thrashing:** The model aggressively evicted useful content, causing the cache to reload data frequently from the backend, increasing latency.

3) *Summary of Tuning Results:* The comparison between Conservative and Aggressive models confirms that for latency-sensitive applications (such as CDNs or databases), the ****Conservative Model**** is the only rational choice. The cost of a cache miss (latency penalty) far outweighs the marginal savings in write operations offered by the Aggressive model. Consequently, the Conservative configuration was selected as the final **PredCLOCK** implementation for the comparative analysis in Table II.

VI. CONCLUSION

Based on the analysis of test data and performance evaluation, the following critical conclusions are drawn regarding the proposed **PredCLOCK** system:

- 1) **Proven Write Efficiency with Conservative Limits:** The implementation of *PredCLOCK* successfully validated the hypothesis that a predictive model can reduce write operations (promotions) to main memory. A consistent reduction of **8–10%** was achieved across all cache size variations compared to the Standard **CLOCK** baseline. While this reduction helps extend the endurance of modern storage devices (such as NVM or SSDs), it falls significantly short of the theoretical

optimal bound of **43%** achieved by the *Offline CLOCK* oracle. This gap indicates that the model, bounded by local feature visibility, adopts a conservative decision-making process to preserve cache hits.

- 2) **Criticality of Handling Class Imbalance:** The parameter tuning phase revealed that the extreme class imbalance in Zipfian workloads (where "one-hit wonders" dominate "reused" objects by approx. 80:20) is the primary challenge for ML-based caching.
 - The **Aggressive Model** (Unweighted Loss) failed systematically, yielding a 2% regression in Miss Ratio (Service Degradation) due to bias toward the majority class.
 - The **Conservative Model** (Weighted Loss 2:1) successfully mitigated this risk, mimicking the safety of Standard CLOCK while filtering obvious waste. This confirms that cost-sensitive learning is mandatory for cache replacement policies.
- 3) **Non-Linear Scalability and Over-Selectivity:** There is a distinct non-linear performance pattern relative to cache size. In high-pressure environments (1% – 20% capacity), *PredCLOCK* provides a "double benefit" of reduced miss ratios and reduced promotions. However, a performance regression (+0.15% Miss Ratio) occurs at a cache size of 40%. At this capacity, the static decision threshold (0.8) becomes **over-selective**, discarding borderline objects that should have been stored given the ample available space. This confirms that static ML thresholds lack the flexibility required for varying memory pressures.
- 4) **Superiority of Oracle Knowledge:** The dominance of the *Optimal Zipf Promotion* strategy over *PredCLOCK* across all metrics confirms that in Zipfian distributions, global knowledge regarding object popularity is far more valuable than the local access patterns (recency and short-term frequency) utilized by *PredCLOCK*.
- 5) **Viability of Hybrid Architecture:** Technically, this research demonstrated the viability of a hybrid simulation framework. The integration of Python-trained models into a high-performance C++ environment via **ONNX (Open Neural Network Exchange)** proved stable and efficient, bridging the gap between modern deep learning tools and operating system simulations.

ACKNOWLEDGMENT

The authors would like to thank CloudLab for providing necessary devices for our experiments.

REFERENCES

- [1] W. A. Wulf and S. A. McKee, "Hitting the memory wall: implications of the obvious," *ACM SIGARCH Computer Architecture News*, vol. 23, no. 1, pp. 20–24, 1995.
- [2] P. Prieto, V. Puente, and J. Gregorio, "Multilevel Cache Modeling for Chip-Multiprocessor Systems," *IEEE Computer Architecture Letters*, 2011.
- [3] P. J. Denning, "The locality principle," *Communications of the ACM*, vol. 48, no. 7, pp. 19–24, 2005.
- [4] M. Herlihy, "The art of multiprocessor programming," *Proceedings of the twenty-fifth annual ACM symposium on Principles of distributed computing (PODC '06)*, 2006.
- [5] G. Eads, "NbQ-CLOCK: A Non-blocking Queue-based CLOCK Algorithm for Web-Object Caching," *International Conference on Internet Computing and Big Data (ICOMP'14)*, 2014.
- [6] S. Jiang, F. Chen, and X. Zhang, "CLOCK-Pro: an effective improvement of the CLOCK replacement," *Proceedings of the 2005 USENIX Annual Technical Conference (USENIX ATC '05)*, 2005. [Online]. Available: <http://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.124.4250>
- [7] S. Bansal and D. S. Modha, "CAR: Clock with Adaptive Replacement," *Proceedings of the 3rd USENIX Conference on File and Storage Technologies (FAST '04)*, 2004. [Online]. Available: <http://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.138.583>
- [8] L. Breslau, P. Cao, F. Li, G. M. Phillips, and S. Shenker, "Web caching and Zipf-like distributions: evidence and implications," *IEEE INFOCOM '99. Conference on Computer Communications*, 1999.
- [9] Q. Chen, M. H. M. Al-Araby, Z. Qiu, Z. Chen, R. Vinayak, and J. Yang, "Demystifying and improving lazy promotion in cache eviction," to appear in *Vldb 2026*.
- [10] J. Yang, Y. Zhang, Z. Qiu, Y. Yue, and R. Vinayak, "FIFO queues are all you need for cache eviction," *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)*, 2023. [Online]. Available: <https://doi.org/10.1145/3600006.3613147>
- [11] W.-F. Lin, D.-Y. Tsai, L. Tang, C.-T. Hsieh, C.-Y. Chou, P.-H. Chang, and L. Hsu, "ONNC: A Compilation Framework Connecting ONNX to Proprietary Deep Learning Accelerators," *2019 IEEE International Conference on Artificial Intelligence Circuits and Systems (AICAS)*, 2019. [Online]. Available: <https://doi.org/10.1109/aicas.2019.8771510>
- [12] S. Jiang and X. Zhang, "Lirs: An efficient low inter-reference recency set replacement policy," *SIGMETRICS*, 2002.
- [13] N. Megiddo and D. Modha, "Arc: A self-tuning, low overhead replacement cache," *USENIX FAST*, 2003.
- [14] Y. Zhang, J. Cai, A. Wildani, and A. Klimovic, "Rethinking web cache design for the AI era," in *Proceedings of the 2025 ACM Symposium on Cloud Computing*, ser. SoCC '25. New York, NY, USA: Association for Computing Machinery, November 2025, pp. 1–8.
- [15] G. Vietri, L. V. Rodriguez, W. A. Martinez, S. Lyons, J. Liu, R. Rangaswami, M. Zhao, and G. Narasimhan, "Driving cache replacement with ml-based lecar," in *USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage)*, 2018.
- [16] D. S. Berger, "Towards Lightweight and Robust Machine Learning for CDN Caching," *Proceedings of the 2018 Workshop on Network Meets AI ML*, 2018.
- [17] J. Yang, Y. Zhang, Z. Qiu, Y. Yue, and K. V. Rashmi, "Fifo queues are all you need for cache eviction," in *Symposium on Operating Systems Principles (SOSP)*, 2023.
- [18] Y. Zhang, J. Yang, Y. Yue, Y. Vigfusson, and K. V. Rashmi, "Sieve is simpler than lru: an efficient turn-key eviction algorithm for web caches," in *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2024.
- [19] J. Tian, G.-T. Bercea, T. D. Le, T. Chen, G. Su, H. Imai, Y. Negishi, A. Leu, K. O'Brien, K. Kawachiya, and A. E. Eichenberger, "Compiling ONNX Neural Network Models Using MLIR," *arXiv preprint*, 2020. [Online]. Available: <http://arxiv.org/abs/2008.08272>
- [20] P. Jajal, W. Jiang, A. Tewari, E. Kocinare, J. Woo, A. Sarraf, Y.-H. Lu, G. K. Thiruvathukal, and J. C. Davis, "Interoperability in Deep Learning: A User Survey and Failure Analysis of ONNX Model Converters," *ACM Transactions on Software Engineering and Methodology*, 2024. [Online]. Available: <https://doi.org/10.1145/3650212.3680374>
- [21] F. Chollet et al., "Keras," <https://keras.io>, 2015.
- [22] J. Yang, Y. Yue, and K. V. Rashmi, "A Large-scale Analysis of Hundreds of In-memory Key-value Cache Clusters at Twitter," *ACM Transactions on Storage (TOS)*, 2021. [Online]. Available: <https://doi.org/10.1145/3468521>
- [23] M. Abadi, A. Agarwal, P. Barham, E. Brevdo, Z. Chen, C. Citro, G. S. Corrado, A. Davis, J. Dean, M. Devin, S. Ghemawat, I. Goodfellow, A. Harp, G. Irving, M. Isard, Y. Jia, R. Jozefowicz, L. Kaiser, M. Kudlur, J. Levenberg, D. Mané, R. Monga, S. Moore, D. Murray, C. Olah, M. Schuster, J. Shlens, B. Steiner, I. Sutskever, K. Talwar, P. Tucker, V. Vanhoucke, V. Vasudevan, F. Viégas, O. Vinyals, P. Warden, M. Wattenberg, M. Wicke, Y. Yu, and X. Zheng, "TensorFlow: Large-scale machine learning on heterogeneous systems,"

- 2015, software available from tensorflow.org. [Online]. Available: <https://www.tensorflow.org/>
- [24] L. A. Belady, "A study of replacement algorithms for a virtual-storage computer," *IBM Systems Journal*, 1966.
- [25] H. Sun, Q. L. Cui, J. Huang, and X. Qin, "NCache: A Machine-Learning Cache Management Scheme for Computational SSDs," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2022.
- [26] H. Wang, X. Yi, P. Huang, B. Cheng, and K. Zhou, "Efficient SSD Caching by Avoiding Unnecessary Writes using Machine Learning," *Proceedings of the 47th International Conference on Parallel Processing*, 2018.
- [27] C. Tang, "Cache system design in the tightly coupled multiprocessor system," *Proceedings of the National Computer Conference*, 1976.
- [28] P. Cunningham, M. Cord, and S. J. Delany, "Supervised Learning," *Machine Learning Techniques for Multimedia*, 2008.